PROYECTO: CALCULADORA DE TASAS

Realizado por:

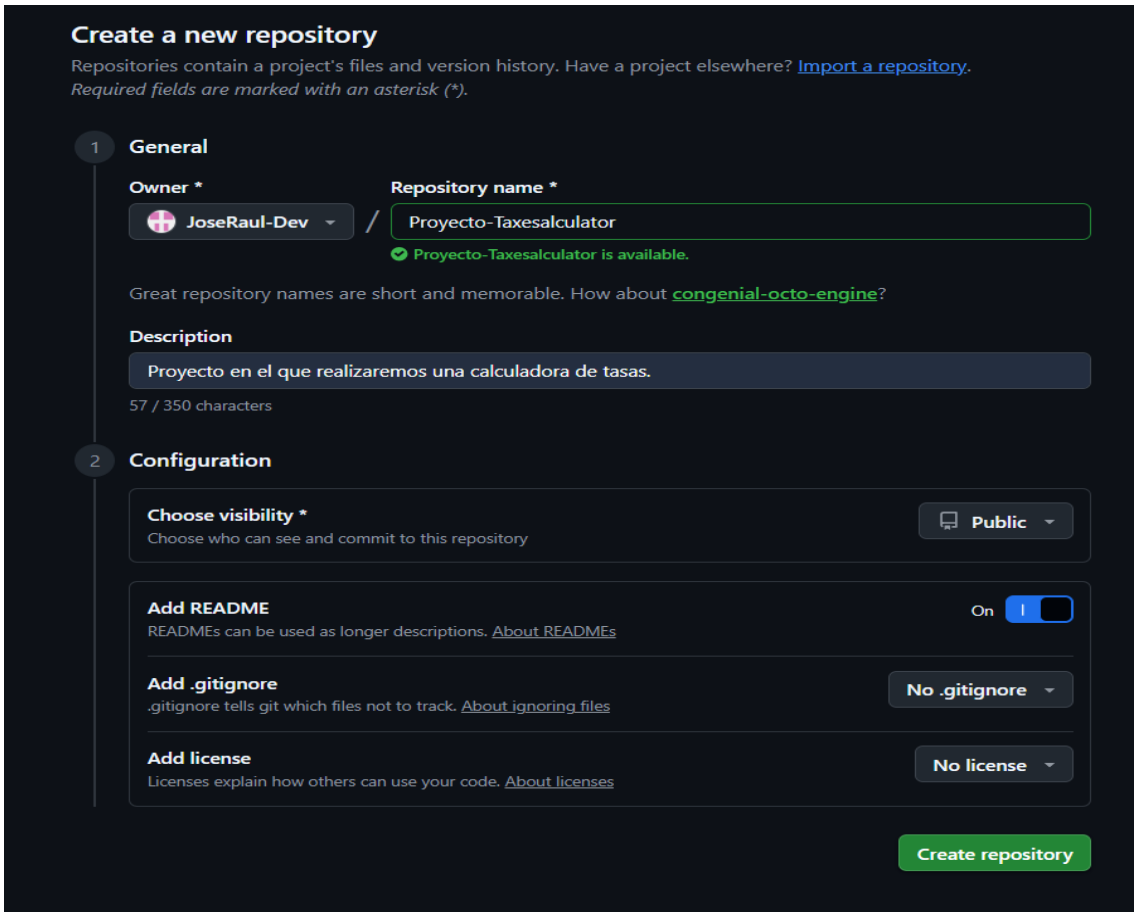
José Raúl Álvarez Rodríguez

Iván Mena Damián

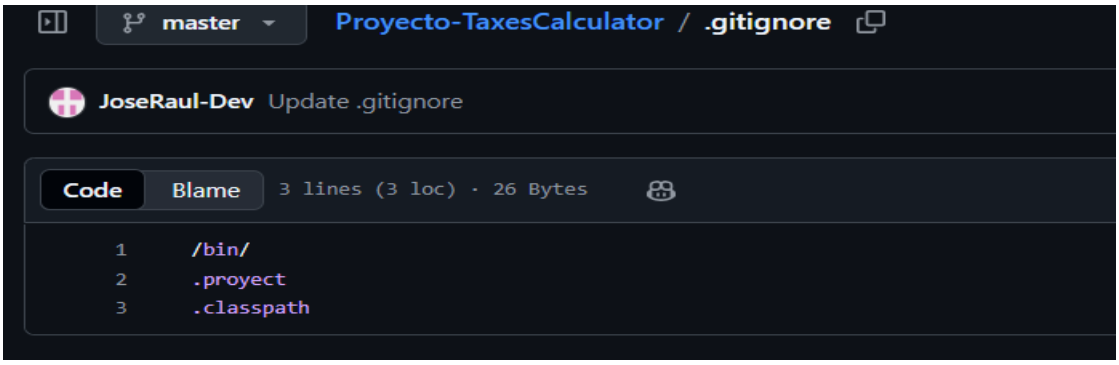
1. Creación del Proyecto y su configuración en GitHub.

1.1. Creación del proyecto.

- Para poder realizar el proyecto, previamente tuvimos que crear el repositorio en GitHub. Para ello, nos dirigimos a nuestro perfil y seleccionamos la opción de crear un nuevo repositorio. En cuanto estemos en la opción de crear el repositorio, se deberá de introducir el nombre, la descripción y seleccionar las opciones que se considere.



- Después de haber creado nuestro proyecto, configuramos el archivo .gitignore. En nuestro caso, optamos por crear un .gitignore adaptado al proyecto, incluyendo los campos que consideramos que podrían encajar en el proyecto.



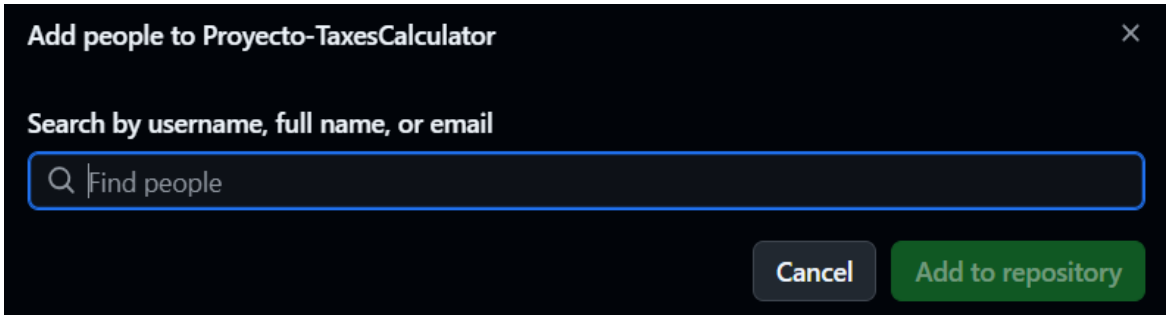
1.2. Gestión de la rama developer

- Para poder trabajar de una manera más organizada, decidimos crear una rama denominada "developer". Esta rama sirve para que cada miembro del equipo pase primero por "developer" antes de pasar a la rama "Master". De esta manera también, evitamos realizar cambios directos sobre la rama principal.

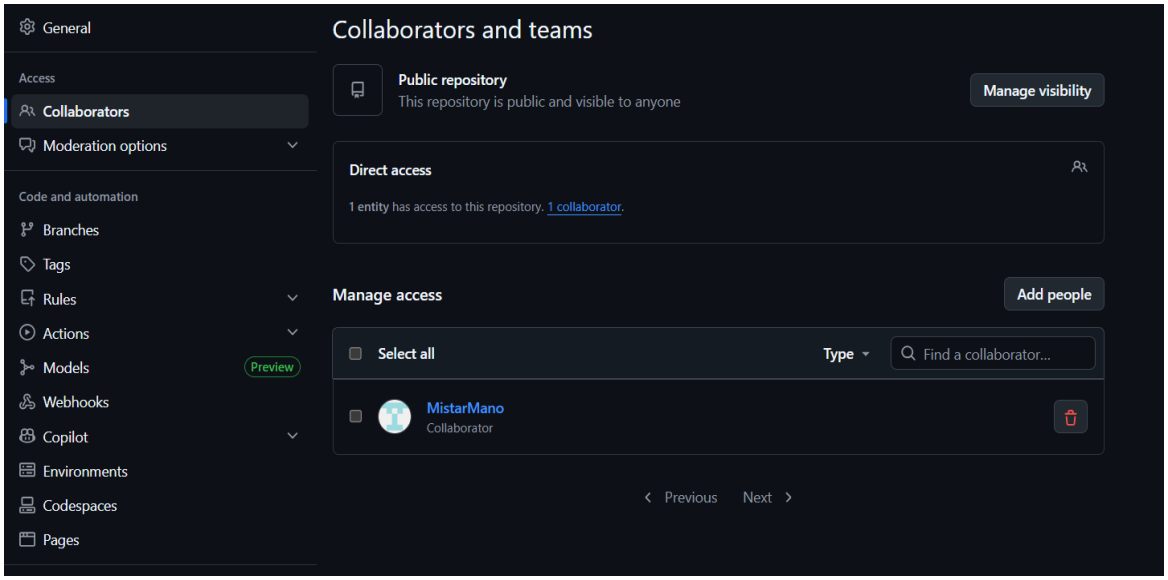
- Cada colaborador en nuestro caso, creo una rama de refactorización por cada una de ellas a partir de la rama "developer".

1.3. Gestión de los colaboradores.

- En el repositorio del proyecto, se encuentran a los siguientes miembros:
- José Raúl Álvarez (Máster) - JoseRaul-Dev.
- Iván Mena (colaborador) - MistarMano.
- Como el repostitorio del proyecto se creo mediante el usuario JoseRaul- Dev, hubo que añadir a MistarMano al mismo. Para ello se invito a MistarMano a que formara parte del mismo. Para ello, hay que dirigirse a "/Settings -> /Collaborators -> /Add people" y después aparecerá el cuadro que se puede observar en la captura.

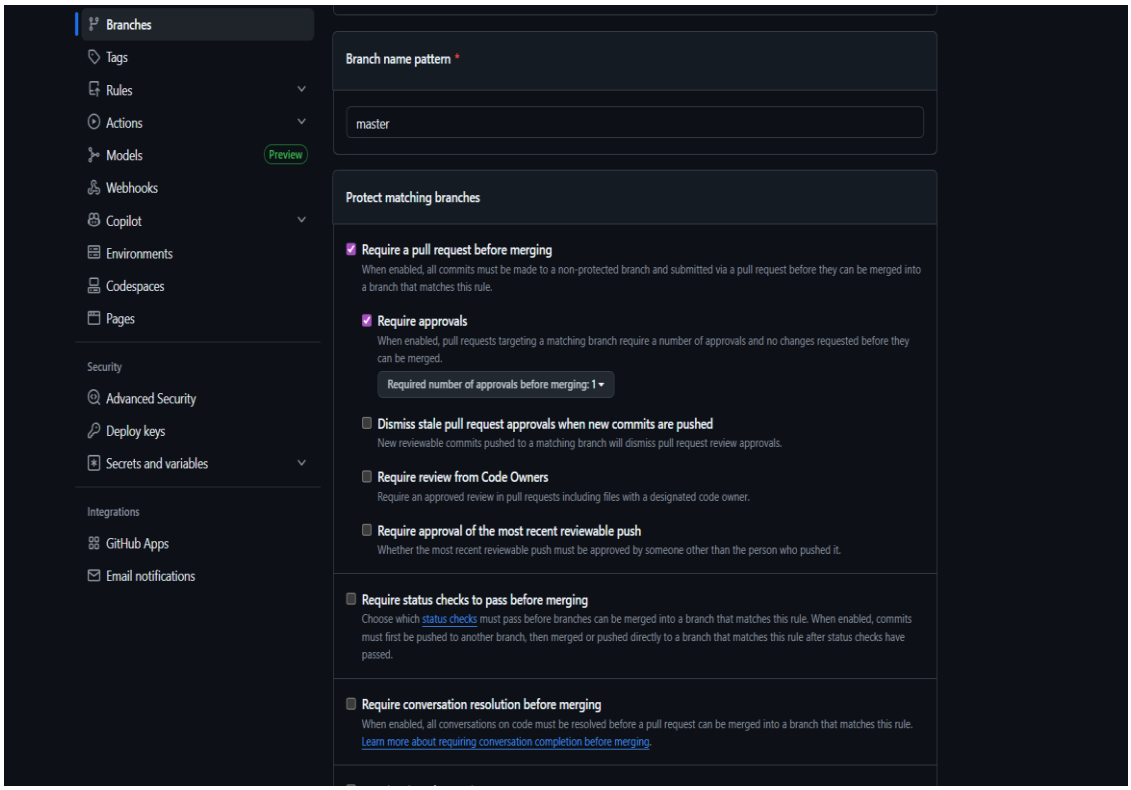


- En cuanto se haya introducido el nombre del usuario y se le haya agregado, deberá aceptar la invitación. Una vez que se aceptó la invitación, el usuario MistarMano se agregó correctamente como colaborador en el repositorio.



1.4. Protección de ramas

- En nuestro repositorio en Git, decidimos que teníamos que proteger nuestra rama "Master" con el objetivo de evitar que haya cualquier tipo de problema, ya que al proteger la rama todo pasa por la aprobación de un colaborador.
- Para ello, accedimos a "/**Settings -> /Branches -> /Add Rule**", donde configuramos una regla de protección para que todos los cambios deban realizarse mediante *pull request* y sean revisados antes de fusionarse con la rama "Master".



2. Realización del proyecto.

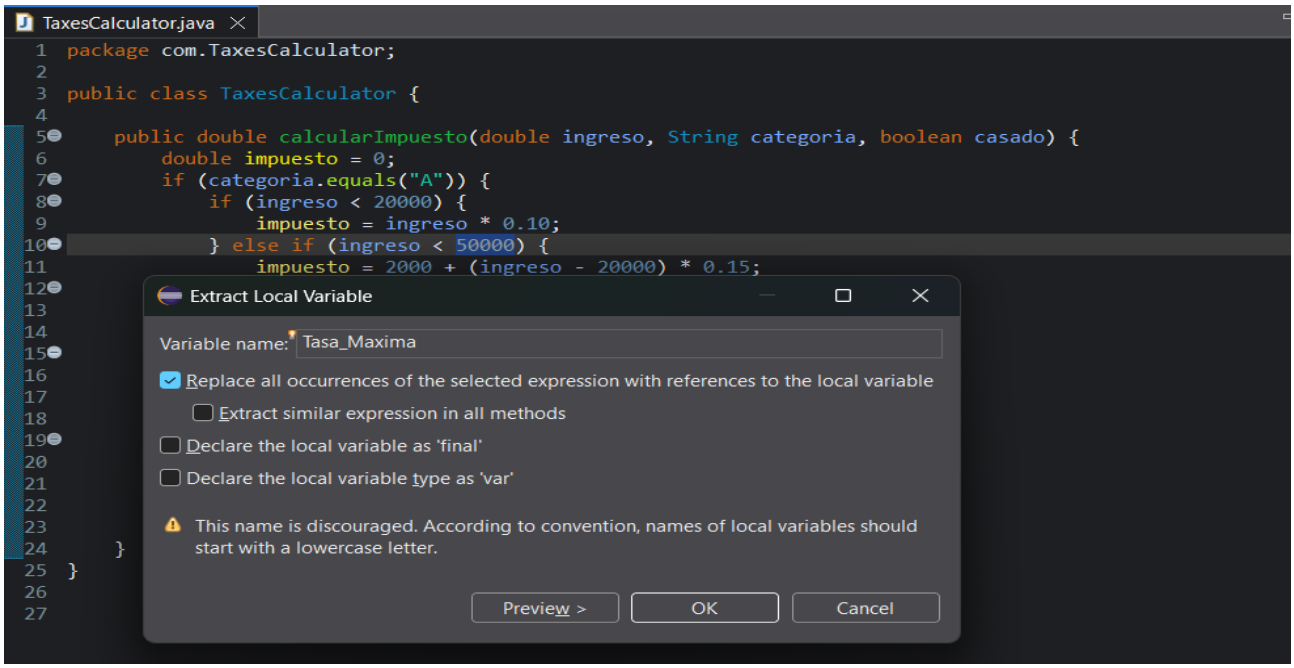
2.1. Análisis del código inicial

- En primer lugar, estuvimos analizando el código inicial que se nos brindó. Durante esta revisión detectamos diferentes problemas relacionados con el código. Estos aspectos que detectamos que no estaban correctamente los hemos corregido en las refactorizaciones que comentamos más adelante.

```java
3   public class TaxesCalculator {
4
5       public double calcularImpuesto(double ingreso, String categoria, boolean casado) {
6           double impuesto = 0;
7           if (categoria.equals("A")) {
8               if (ingreso < 20000) {
9                   impuesto = ingreso * 0.10;
10              } else if (ingreso < 50000) {
11                  impuesto = 2000 + (ingreso - 20000) * 0.15;
12              } else {
13                  impuesto = 6500 + (ingreso - 50000) * 0.25;
14              }
15          } else if (categoria.equals("B")) {
16              impuesto = ingreso * 0.20;
17          }
18
19          if (casado) {
20              impuesto = impuesto * 0.95;
21          }
22
23          return impuesto;
24      }
25  }
```

2.2. Primera Refactorización

- En esta primera refactorización, encontramos ciertos valores que se repetían de forma constante. Por ese motivo, decidimos extraer esos campos y convertirlos en constantes para que en el caso de que en el futuro se quisieran cambiar, podríamos hacerle desde un único lugar. A estos campos se les asignaron los nombres de “Tasa_Maxima, Tasa_Minima, Base_Media y Base Alta”. Para ello utilizamos la herramienta que nos proporciona el IDE de “/Refactor -> Extract Constant”.



```
*TaxesCalculator.java
1  package com.TaxesCalculator;
2
3  public class TaxesCalculator {
4      final int Tasa_Maxima = 50000;
5
6      public double calcularImpuesto(double ingreso, String categoria, boolean casado) {
7          double impuesto = 0;
8          if (categoria.equals("A")) {
9              if (ingreso < 20000) {
10                 impuesto = ingreso * 0.10;
11             } else {
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27     }
28 }
29
```

Extract Local Variable

Variable name: Tasa_Minima

- [x] Replace all occurrences of the selected expression with references to the local variable
- [ ] Extract similar expression in all methods
- [x] Declare the local variable as 'final'
- [ ] Declare the local variable type as 'var'

This name is discouraged. According to convention, names of local variables should start with a lowercase letter.

Preview > | OK | Cancel

```
3  public class TaxesCalculator {
4
5      private static final int Tasa_Maxima = 50000;
6      private static final int Tasa_Minima = 20000;
7
8
9
10
11     public double calcularImpuesto(double ingreso, String categoria, boolean casado) {
12         double impuesto = 0;
13         if (categoria.equals("A")) {
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
```

Extract Local Variable

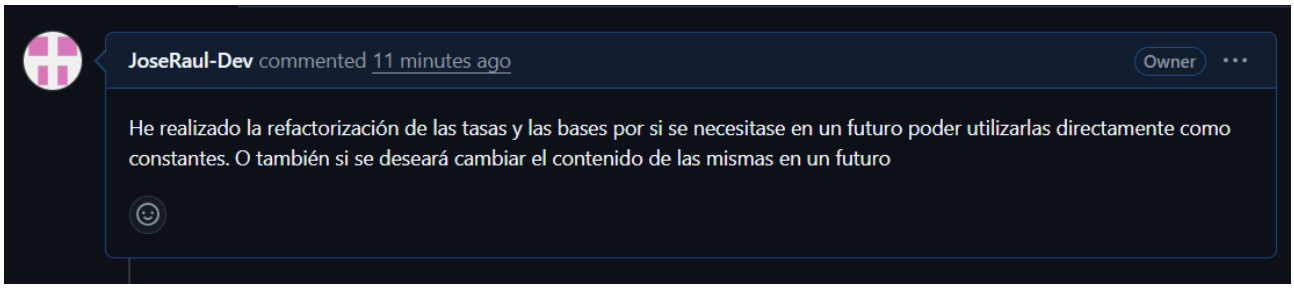
Variable name: Base_Media

- [x] Replace all occurrences of the selected expression with references to the local variable
- [ ] Extract similar expression in all methods
- [x] Declare the local variable as 'final'
- [ ] Declare the local variable type as 'var'

This name is discouraged. According to convention, names of local variables should start with a lowercase letter.

Preview > | OK | Cancel

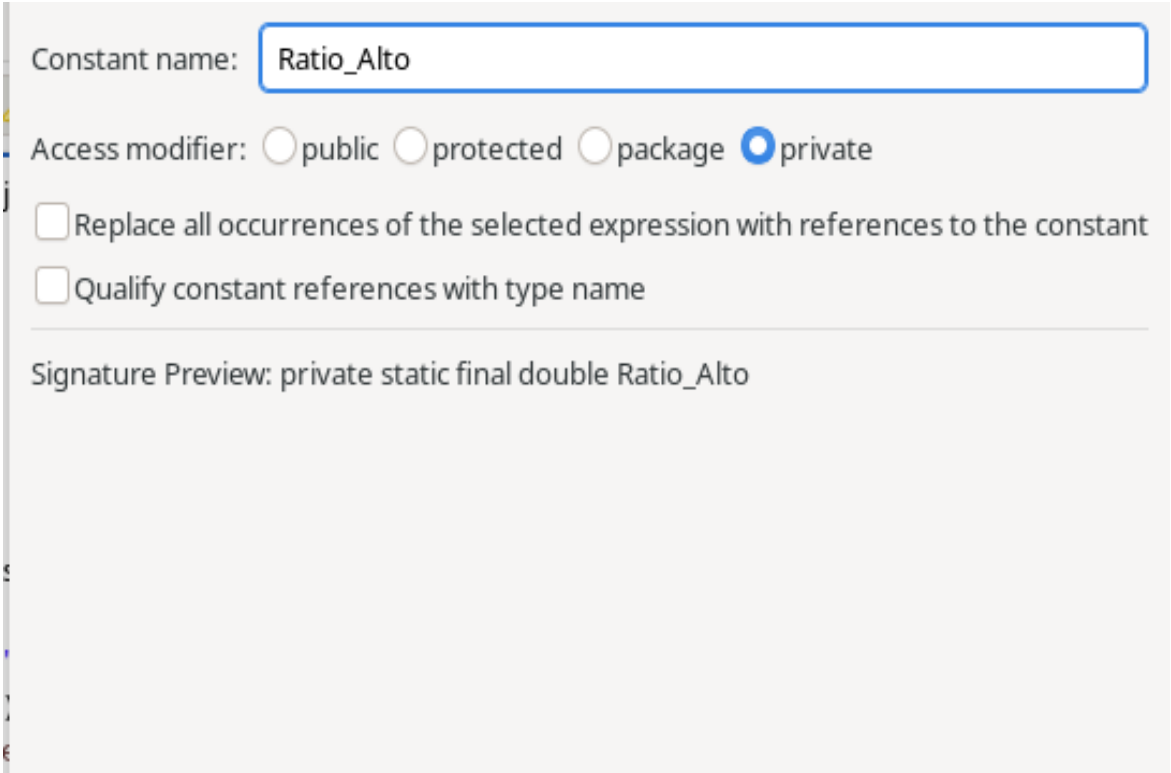
- Además, al realizar el commit correspondiente a nuestro repositorio, añadimos un comentario explicando los cambios que habíamos realizado.

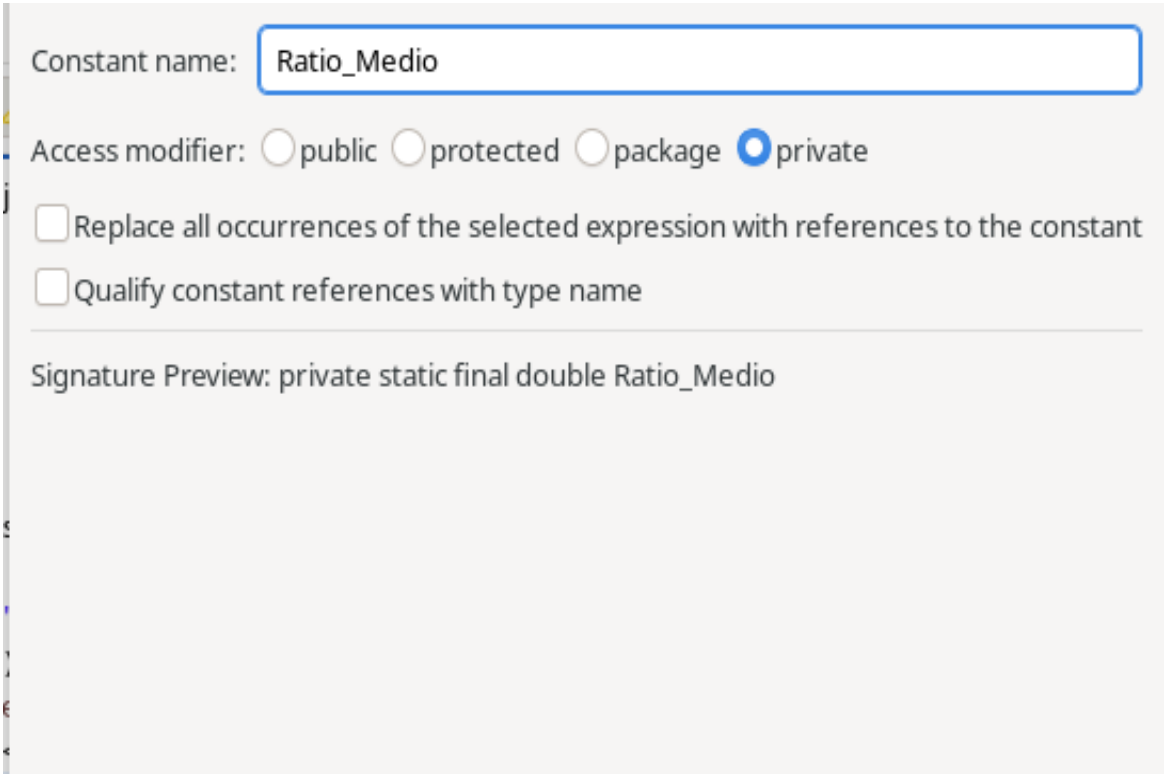


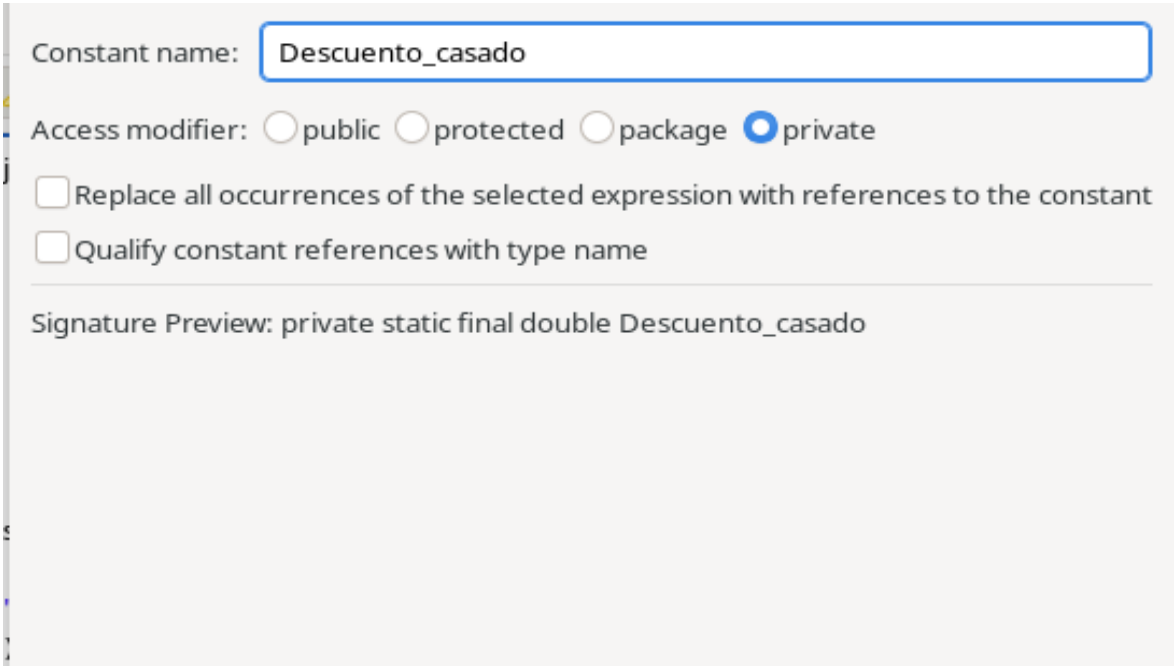
- En cada commit que realizamos al repositorio, introdujimos una descripción del trabajo que se había realizado.

2.3. Segunda Refactorización

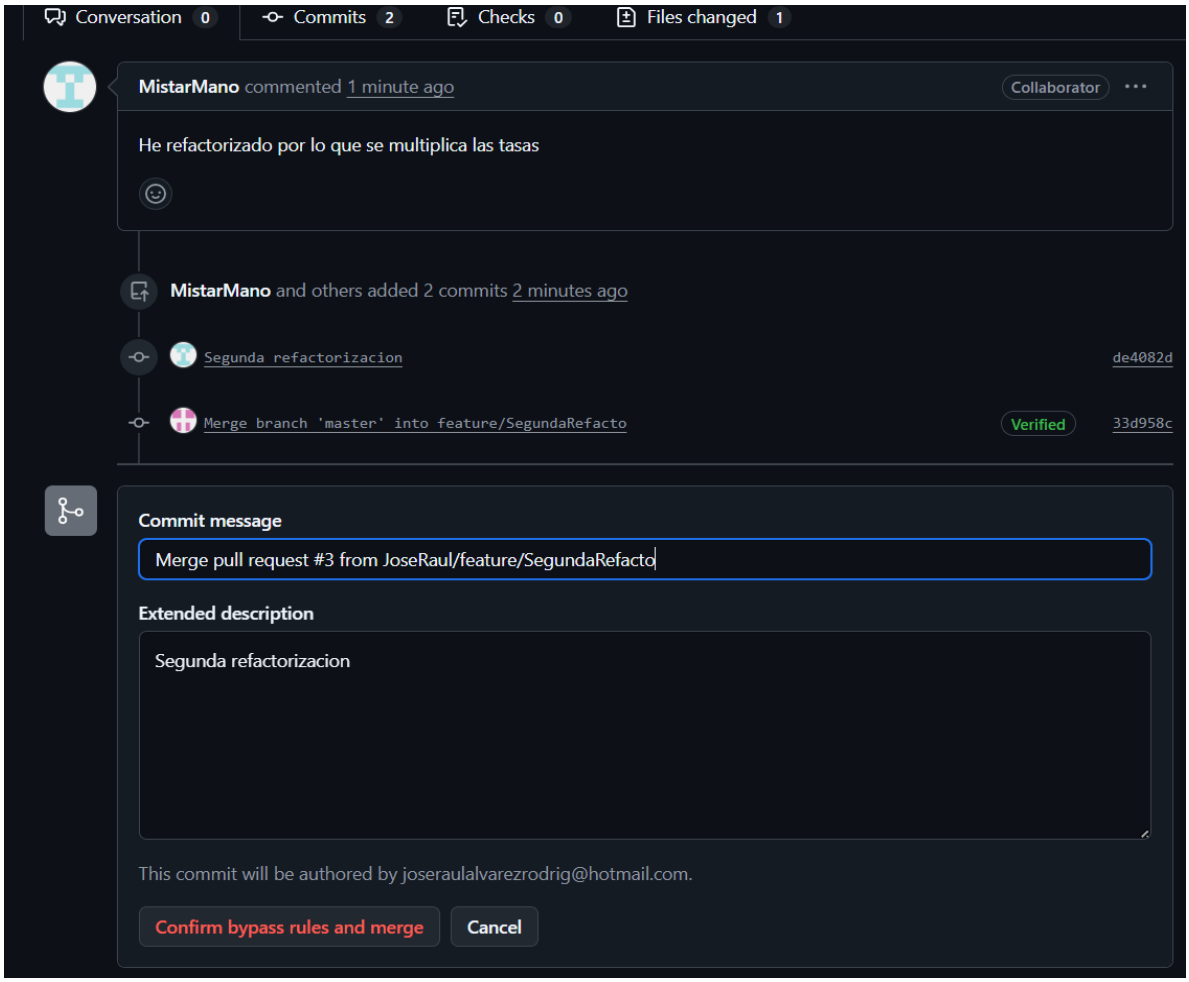
- En esta segunda refactorización, decidimos extraer también los porcentajes aplicados según el nivel de tasa (alto, medio o bajo), así como el descuento aplicado en caso de estar casado.
- Como en la refactorización anterior, convertimos dichos porcentajes en constantes para que pudieran modificarse fácilmente y así poder tener el código de una manera más limpia y organizada. Para ello utilizamos la herramienta que nos proporciona el IDE de "/Refactor -> Extract Constant".





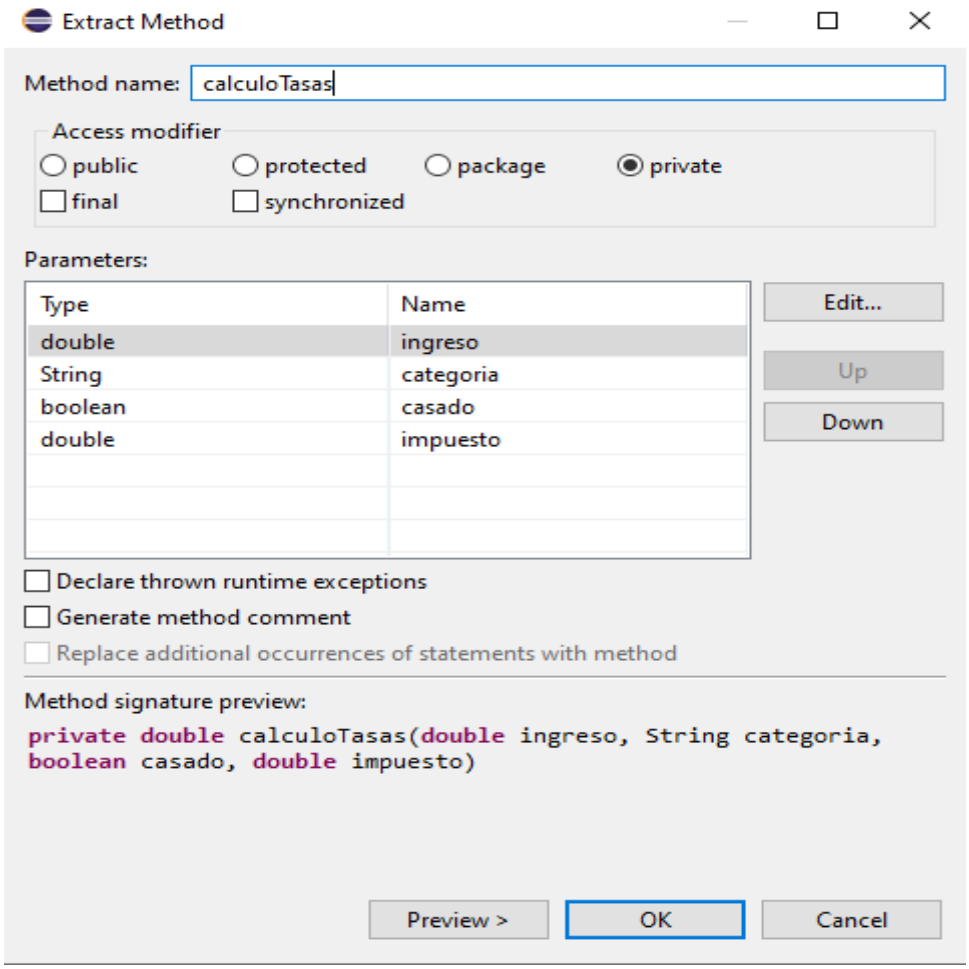


- Como se hizo en el paso anterior dejamos un comentario describiendo lo que habíamos realizado a la hora de realizar el commit.

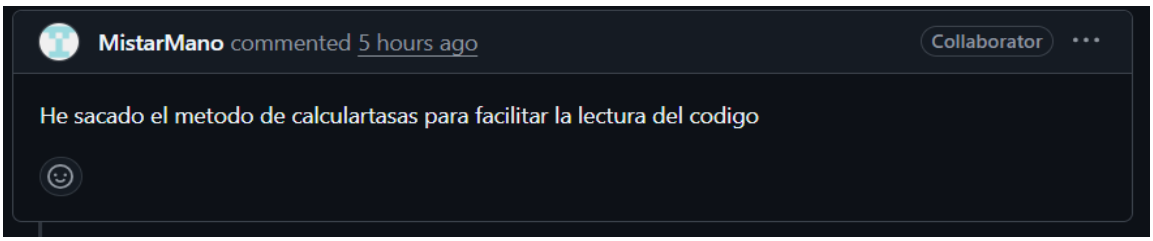


2.4. Tercera Refactorización

- En esta refactorización observamos que se podría extraer el método para dejar nuestro método principal de una forma más clara y limpia y así no tenerlo tan sobrecargado. Por ello, decidimos extraer parte del código a otro método diferente. Para realizar esta refactorización utilizamos la herramienta que nos proporciona el IDE de “/Refactor -> /Extract Method”.

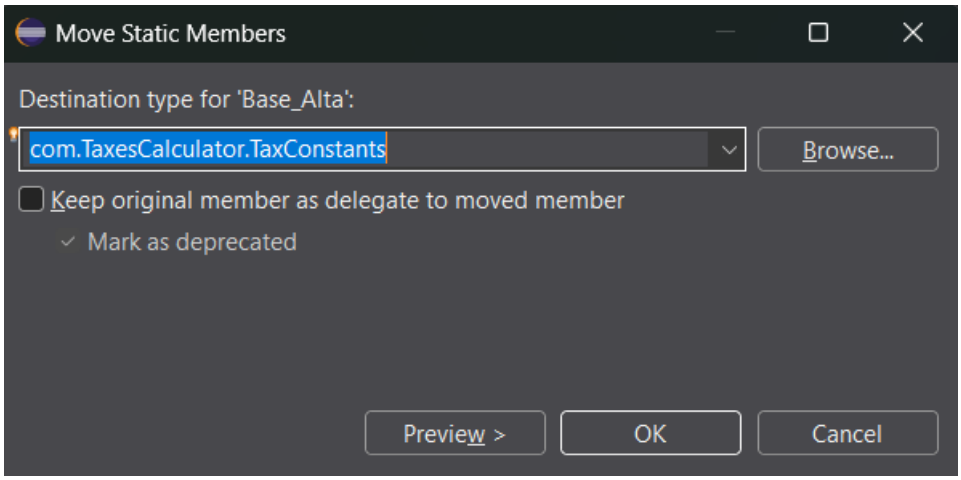


- El nuevo método que creamos se encarga de realizar los cálculos necesarios y devolver el resultado al método principal.
- Posteriormente, como en las anteriores refactorizaciones se introdujo un comentario dejando constancia de los cambios realizados al realizar el commit.

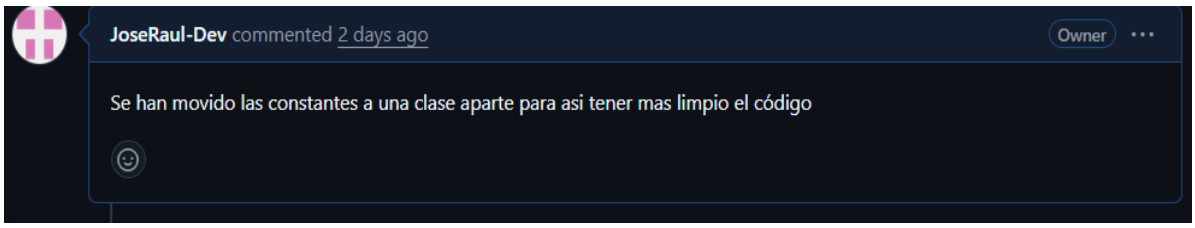


2.5. Cuarta Refactorización

- En esta última refactorización, consideramos que teníamos que mover nuestras constantes a otra clase para así poder tener la clase principal más limpia, ordenada y estructura. Para ello se decidió llamar a la otra clase “TaxConstants”. Después de crearla, movimos cada una de las constantes a dicha clase. Para ello utilizamos la herramienta que nos proporciona el IDE de “/Refactor -> /Move”.



- Como en los casos anteriores, añadimos un comentario explicando los cambios que se realizaron cuando realizamos el commit.



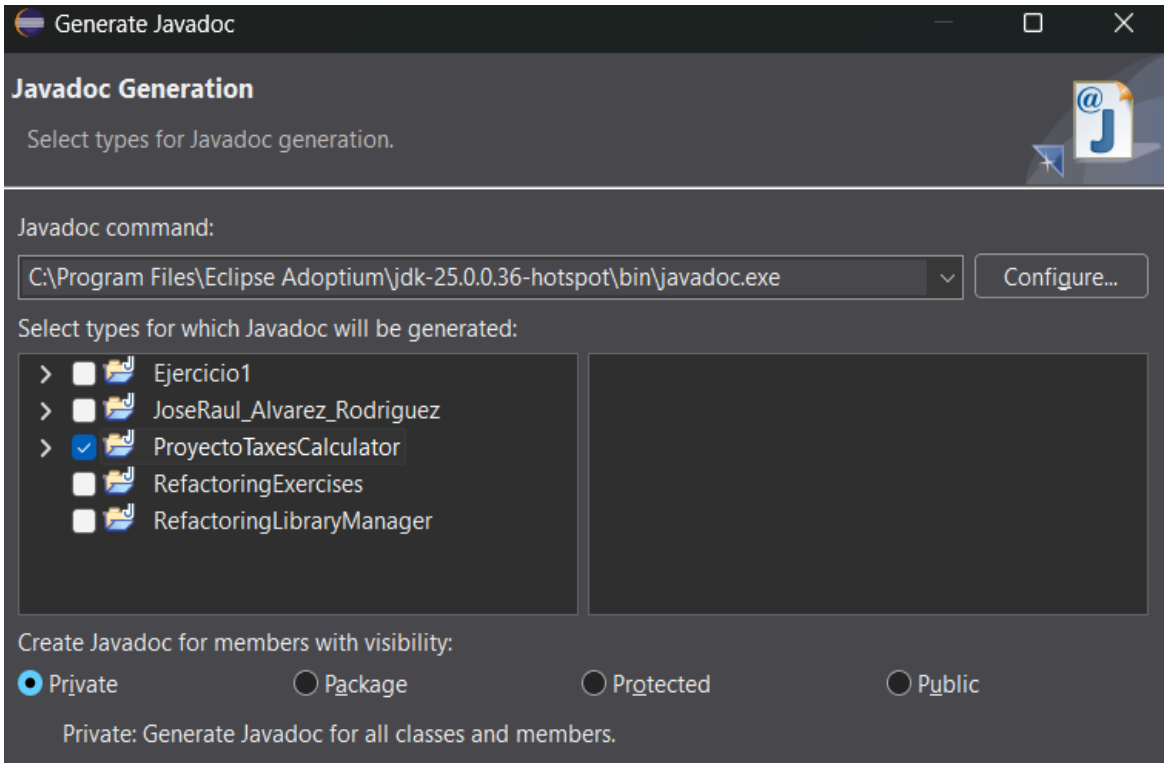
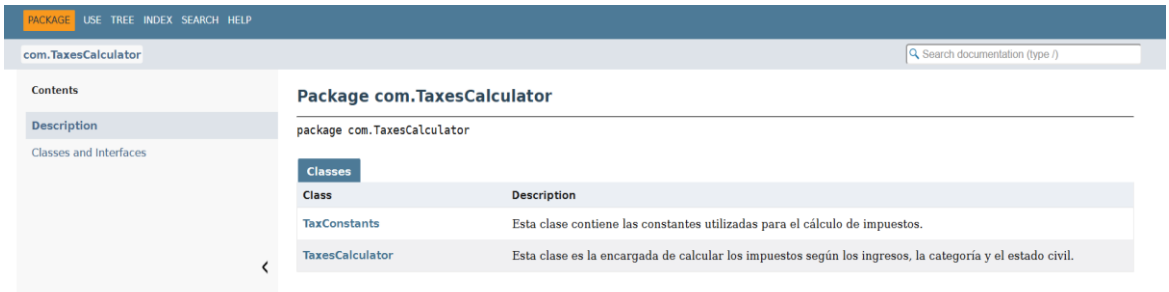
3. Creación del documento html "JavaDoc".

3.1. Comentarios en el código.

- Para poder generar la documentación JavaDoc, fue necesario añadir comentarios específicos en el código utilizando la sintaxis /** */. Incluimos los comentarios que consideramos que son importantes para la correcta creación de la documentación. También, es importante que, si no se realizan comentarios, JavaDoc no puede generar correctamente la documentación.

3.2. Creación de JavaDoc.

- Posteriormente, cuando añadimos los comentarios necesarios, procedimos a generar dicha documentación. Seleccionamos el proyecto correspondiente y generamos dicha documentación.



- Finalmente, revisamos el documento HTML generado para comprobar que toda la información estuviera correctamente realizada.